

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO



**FACULTAD DE CIENCIAS DE LA INGENIERÍA DE LA COMPUTACIÓN
CARRERA DE INGENIERÍA EN SOFTWARE**

TEMA:

PRÁCTICA EXPERIMENTAL UNIDAD III

MATERIA:

INGENIERÍA DE REQUERIMIENTOS

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERÓN

INTEGRANTES:

CEDEÑO CORONADO WILSON LIZANDRO, CI: 1206867200, wcedenoc2@uteq.edu.ec

MORALES SANCHEZ GARY ALEJANDRO, CI: 1251154173, gmoraless2@uteq.edu.ec

SANCHEZ CORNEJO GARY ALBERTO, CI: 1208338291, gsanchez6@uteq.edu.ec

CURSO:

CUARTO SEMESTRE PARALELO “B”

Repositorio GitHub:

https://github.com/MoralesGaryUteq/PracticaExperimental3_GrupoD

QUEVEDO – LOS RIOS – ECUADOR

2026 – 2027 PPA

1. Introducción

La Ingeniería de Requisitos constituye una de las disciplinas fundamentales del proceso de desarrollo de software, dado que de su correcta ejecución depende la calidad, viabilidad y trazabilidad del producto final. Su objetivo es transformar las necesidades expresadas por los stakeholders en especificaciones precisas, no ambiguas y verificables, siguiendo un proceso sistemático de elicitación, análisis, especificación y validación, conforme a lo establecido en IEEE/ISO/IEC 29148:2018 [1].

El presente documento corresponde a la tercera Práctica Experimental (PE3) de la asignatura Ingeniería de Requerimientos, y tiene como propósito construir los modelos UML de especificación de requisitos del Proyecto de Fin de Carrera [nombre del PFC: MundiPets] (desde las perspectivas de datos, funcional y de comportamiento) utilizando herramientas CASE, e integrarlos en un documento ERS/SRS estructurado según la norma IEEE/ISO/IEC 29148:2018 [1]. Este trabajo parte de la lista de requisitos depurada en la Práctica Experimental 2 (PE2) y del SRS parcial elaborado en la Entrega 1B, sobre los cuales se aplica un proceso de revisión, corrección y modelado formal.

El documento se organiza en siete secciones principales. La sección 2 desarrolla el fundamento teórico que sustenta el proceso de especificación de requisitos, abarcando el marco normativo aplicable, la documentación de requisitos y las tres perspectivas de modelado (datos, función y comportamiento) junto con sus interrelaciones. La sección 3 presenta la revisión y refinamiento del SRS parcial, incluyendo el registro de defectos detectados, su corrección mediante sintaxis EARS y la tabla de atributos completa para los requisitos funcionales y no funcionales. Las secciones 4, 5 y 6 desarrollan, respectivamente, el modelo de datos (diagrama Entidad-Relación y diagrama de clases UML), el modelo funcional (diagramas de flujo de datos y diagrama de actividades UML) y el modelo de comportamiento (máquina de estados UML y diagrama de secuencia UML). Finalmente, la sección 7 integra los tres modelos mediante una matriz de trazabilidad, una tabla comparativa y las conclusiones sobre la complementariedad de las perspectivas abordadas.

Este enfoque multiperspectiva responde a la necesidad, señalada por Pohl [2], de verificar la consistencia de los requisitos desde distintos ángulos de análisis, dado que ninguna perspectiva de modelado resulta suficiente por sí sola para garantizar la completitud y coherencia de una especificación de software.

2. Fundamento teórico

2.1. Marco normativo aplicado a la especificación de requisitos

La ingeniería de requisitos se sustenta en un conjunto de normas internacionales que definen el ciclo de vida, los procesos y los artefactos documentales del software. La norma IEEE/ISO/IEC 29148:2018 [1] constituye el referente principal para la especificación de requisitos, al definir cuatro tipos de documentos según el nivel de abstracción: el Stakeholder

Requirements Specification (StRS), el System Requirements Specification (SyRS), el Software Requirements Specification (SRS) y el Operational Concept Description (OpsCon). Esta norma reemplaza formalmente a IEEE 830:1998, ampliando su alcance desde una simple guía de contenido hacia un marco integral de procesos de requisitos.

La ingeniería de requisitos ha evolucionado progresivamente desde métodos tradicionales hacia enfoques ágiles, con un énfasis creciente en la automatización de sus actividades mediante herramientas de procesamiento de lenguaje natural y verificación automática de calidad [3].

La trazabilidad entre los requisitos de sistema y los de software, exigida en el Paso 5 de esta práctica, encuentra su fundamento en IEEE/ISO/IEC 15288:2023 [4], que establece los procesos de ciclo de vida del sistema y define explícitamente el proceso de gestión de requisitos como puente entre las necesidades del stakeholder y las especificaciones técnicas. De manera complementaria, IEEE/ISO/IEC 12207:2017 [5] particulariza estos procesos para el ciclo de vida del software, delimitando las actividades de análisis de requisitos dentro del proceso de desarrollo.

La calidad de un documento SRS (completitud, consistencia, verificabilidad y trazabilidad) se evalúa con el modelo de calidad de producto de software definido en ISO/IEC 25010:2023 [6], el cual será aplicado en el Paso 1 de esta práctica para la detección de defectos en el SRS parcial de la Entrega 1B.

2.2. Documentación de requisitos

Documentar un requisito no equivale únicamente a redactarlo, sino a garantizar que cumpla atributos de calidad verificables. Pohl [2], en la Parte IV de su obra, caracteriza los artefactos de requisitos como el resultado de un proceso sistemático de elicitación, especificación y validación, distinguiendo entre requisitos funcionales, no funcionales y restricciones. El SWEBOK v4.0 [7], en su capítulo 1 (secciones 1.5–1.7), refuerza este enfoque al establecer que la documentación de requisitos debe ser trazable, priorizable y verificable, además de mantenerse bajo control de cambios durante todo el ciclo de vida del proyecto.

Desde la perspectiva de gestión, el PMBoK 7.^a ed. [8] enmarca la documentación de requisitos como un entregable clave del dominio de desempeño de planificación, subrayando la importancia de la trazabilidad entre requisitos y objetivos del proyecto. Finalmente, IREB CPRE FL v3.1 [9] aporta los criterios de calidad a nivel de requisito individual (no ambiguo, verificable, necesario, factible y priorizado), criterios que se aplican directamente en el registro de defectos del Paso 1 de esta práctica.

Para el caso del PFC MundiPets, la documentación de requisitos se organiza siguiendo estos criterios de calidad, como se evidencia en la tabla de atributos del Anexo A, donde cada requisito funcional y no funcional se clasifica según su fuente, estabilidad y verificabilidad.

2.3. Documento ERS/SRS según IEEE/ISO/IEC 29148:2018

La norma IEEE/ISO/IEC 29148:2018 [1], en su sección 9, define la estructura recomendada para un documento SRS, compuesta por secciones como introducción, descripción general, requisitos específicos (funcionales, de interfaz, de rendimiento, de calidad de software) y anexos. Esta estructura busca garantizar que el documento sea autocontenido, no ambiguo y verificable por las partes interesadas.

Estudios recientes refuerzan la importancia de estructurar el SRS mediante plantillas normalizadas (boilerplates), demostrando que este enfoque reduce la ambigüedad y mejora la trazabilidad de los requisitos frente a especificaciones redactadas en texto libre [10].

Hull, Jackson & Dick [11] complementan esta visión al señalar que un SRS eficaz debe equilibrar el nivel de detalle con la trazabilidad hacia los requisitos de sistema de nivel superior, evitando tanto la subespecificación como el sobrediseño prematuro (gold plating). Asimismo, indican que la calidad de un SRS depende en gran medida de la consistencia terminológica y de la ausencia de requisitos duplicados o contradictorios, aspectos verificados en el Paso 1 mediante el registro de defectos.

Para el PFC, el SRS parcial de la Entrega 1B se estructura conforme a estas directrices, incorporando en esta entrega la revisión de defectos, la corrección mediante sintaxis EARS y la tabla de atributos completa, en cumplimiento de los requisitos de calidad establecidos en ISO/IEC 25010:2023 [6].

2.4. Modelos de datos (perspectiva de datos)

La perspectiva de datos responde a la pregunta de qué información gestiona el sistema, independientemente de los procesos que la transforman. Maciaszek [12] define el diagrama Entidad-Relación (ER) como una técnica de modelado conceptual que representa entidades, atributos y relaciones con su cardinalidad y participación, y establece la Tercera Forma Normal (3FN) como criterio de calidad para evitar redundancias y anomalías de actualización.

El SWEBOK v4.0 [7] ubica el modelado de datos dentro de las técnicas de análisis de requisitos orientadas a la comprensión del dominio del problema, mientras que Pohl [2] enfatiza que el modelo de datos constituye la base para la posterior transformación hacia el diagrama de clases UML, en el cual las entidades se convierten en clases con atributos tipados, visibilidad y operaciones.

En el PFC, las entidades identificadas a partir de los sustantivos clave de los requisitos funcionales (detalladas en la sección 4) se modelan siguiendo la notación Crow's Foot y se transforman en un diagrama de clases UML que incorpora clases asociativas para las relaciones muchos a muchos, conforme a los lineamientos de Maciaszek [12].

2.5. Modelos funcionales (perspectiva funcional)

La perspectiva funcional describe las transformaciones que el sistema realiza sobre los datos. Maciaszek [12] define el Diagrama de Flujo de Datos (DFD) como una técnica jerárquica de descomposición funcional, donde el Nivel 0 (diagrama de contexto) representa el sistema como un único proceso frente a sus entidades externas, y el Nivel 1 descompone dicho proceso en subprocesos con sus respectivos almacenes de datos, manteniendo el principio de balanceo entre niveles. El DFD esencial, de acuerdo con Maciaszek, se centra en el proceso de negocio más relevante, abstrayendo detalles de implementación.

Desde una perspectiva orientada a objetos, Pohl [2] describe el diagrama de actividades UML como el complemento dinámico del DFD, permitiendo representar flujos de trabajo con particiones por actor (swimlanes), nodos de decisión y bifurcaciones/uniones concurrentes (fork/join), así como flujos de objeto que enlazan explícitamente con las clases del modelo de datos.

Para el PFC, el proceso de negocio principal (identificado en el diagrama de contexto de la sección 5) se descompone en al menos cinco subprocesos en el DFD Nivel 1, y su lógica se representa adicionalmente mediante un diagrama de actividades UML del caso de uso principal.

2.6. Modelos de comportamiento (perspectiva de comportamiento)

La perspectiva de comportamiento modela cómo cambian de estado las entidades del sistema a lo largo del tiempo. Harel [13], en su formalismo de statecharts, introduce los conceptos de estados compuestos (jerárquicos) y transiciones etiquetadas con la notación evento[condición]/acción, formalismo que constituye la base directa de la máquina de estados UML utilizada en esta práctica. Este enfoque permite representar comportamientos complejos de manera más compacta que las máquinas de estado finito tradicionales, al admitir concurrencia y anidamiento de estados.

Pohl [2] complementa este modelo con el diagrama de secuencia UML, el cual documenta la interacción temporal entre objetos mediante líneas de vida, mensajes síncronos y asíncronos, y fragmentos combinados como alt (alternativa condicional) y loop (iteración), útiles para especificar escenarios principales y alternativos de un caso de uso.

En el PFC, la entidad con mayor número de estados en su ciclo de vida (identificada en la sección 6) se modela mediante una máquina de estados UML con al menos un estado compuesto, siguiendo el formalismo de Harel [13], y su escenario principal se documenta mediante un diagrama de secuencia con fragmentos alt y loop.

2.7. Interrelaciones entre las tres perspectivas

Ninguna de las tres perspectivas (datos, función y comportamiento) resulta suficiente por sí sola para especificar un sistema de software completo. Pohl [2] sostiene que estas perspectivas son complementarias y deben verificarse cruzadamente: las entidades del

modelo de datos deben aparecer consistentemente como almacenes o flujos en el modelo funcional, y como objetos con ciclo de vida en el modelo de comportamiento. El SWEBOK v4.0 [7] refuerza esta idea al señalar que la validación de requisitos requiere la triangulación entre múltiples vistas del sistema, dado que cada perspectiva revela defectos u omisiones que las otras no evidencian.

IEEE/ISO/IEC 29148:2018 [1] formaliza esta necesidad de coherencia a través del concepto de trazabilidad bidireccional, exigiendo que cada requisito funcional esté cubierto por al menos un elemento de modelado en las tres perspectivas, y que todo elemento de modelado se justifique en al menos un requisito, evitando así tanto los vacíos de cobertura (gaps) como el sobre-diseño (gold plating).

Para el PFC, esta interrelación se documenta explícitamente en la matriz de trazabilidad de la sección 7, donde se verifican cruces entre el diagrama ER y el diagrama de clases, entre las clases UML y los procesos del DFD, y entre los estados de la máquina de estados y las actividades del flujo de trabajo principal.

3. Revisión y refinamiento del SRS parcial

3.1. Registro de defectos del SRS

3.2. Corrección con sintaxis EARS

3.3. Tabla de atributos completa (RF y RFN)

4. Modelo de datos: ER y clases UML

4.1. Identificación de entidades y relaciones

4.2. Diagrama Entidad-Relación (Crow's Foot)

4.3. Diagrama de clases UML

5. Modelo funcional: DFD y actividades UML

5.1. Diagrama de contexto (DFD Nivel 0)

5.2. DFD nivel 1 y DFD esencial

5.3. Diagrama de actividades UML

6. Modelo de comportamiento: estados y secuencia

6.1. Diagrama de máquina de estados UML

6.2. Diagrama de secuencia UML

7. Matriz de trazabilidad y tabla comparativa

7.1. Matriz de trazabilidad

RF	CU	Clases UML	Procesos DFD	Actividades UML	Estados UML
RF-01 Registrar mascota	CU-01	Mascota, Fotografía, Propietario	P2	✓	—
RF-02 Gestionar historial médico	CU-02, CU-03	HistorialMedico, DocumentoVeterinario	P3	—	—
RF-03 Consultar	CU-10	Mascota, HistorialMedico	P2	—	—

perfil completo					
RF-04 Buscar y filtrar mascotas	CU-11	Mascota	P2	—	—
RF-05 Gestionar solicitudes de adopción	CU-07	Solicitud, Publicacion, Usuario	P5	—	✓
RF-06 Evaluar compatibilidad para cruce	CU-13	EvaluacionCompatibilidad, Solicitud	P6	—	—
RF-07 Mensajería entre usuarios	CU-08	Mensaje	—	—	—
RF-08 Gestionar solicitudes de cruce	sin CU propio	Solicitud	P5	—	~
RF-09 Validar información médica	CU-09	HistorialMedico, DocumentoVeterinario, Veterinario, EvaluacionVeterinaria	P3	—	—
RF-10 Recordatorios de controles preventivos	CU-12	Vacuna	—	—	—
RF-11 Administrar privacidad	CU-06	ConfiguracionPrivacidad, Usuario	P1	—	—
RF-12 Verificar identidad de usuarios	sin CU propio	Usuario (estadoVerificacion)	P1	—	—

RF-13 Registrar publicaciones	CU-04, CU-05	Publicacion	P4	—	—
RF-14 Gestionar carnet de vacunación	sin CU propio	Vacuna	vía P3	—	—
RF-15 Consultar historial de procesos	sin CU propio	Solicitud	vía P5	—	—
RF-16 Antecedentes genéticos y parentesco	CU-02, CU-13	Mascota, AntecedenteGenetico	P2	—	—
RF-17 Validar imágenes	sin CU propio	Fotografía (procesarValidacionIA())	vía P6	—	—

Gaps identificados (RF sin cobertura completa):

1. Cinco requisitos funcionales no cuentan con un caso de uso formalizado en el documento de casos de uso: **RF-08** (Gestionar solicitudes de cruce), **RF-12** (Verificar identidad de usuarios), **RF-14** (Gestionar carnet de vacunación), **RF-15** (Consultar historial de procesos) y **RF-17** (Validar imágenes mediante IA), aunque sí se encuentran cubiertos por el modelo de clases y por procesos del DFD Nivel 1. Se recomienda evaluar si corresponden a funciones de validación automática que no requieren una interacción explícita del usuario (como RF-12 y RF-17), o si es necesario redactar los casos de uso correspondientes para completar la trazabilidad.
2. El requisito **RF-10** (Recordatorios de controles preventivos) se encuentra solo parcialmente cubierto por **CU-12** (Consultar recomendaciones de cuidado): este último describe una consulta bajo demanda del usuario, mientras que RF-10 exige que el sistema genere y envíe recordatorios de forma proactiva. Se recomienda formular un caso de uso adicional, del tipo "Enviar recordatorio de control preventivo", con el propio sistema como actor iniciador.

3. El requisito **RF-07** (Mensajería entre usuarios) no cuenta con un proceso dedicado dentro del DFD Nivel 1, pese a estar cubierto por la clase Mensaje en el modelo de datos.

Gold plating identificado (elementos del modelo sin requisito asociado):

1. La clase Administrador, con su atributo nivelAcceso y sus operaciones gestionarUsuarios() y asignarRol(), no corresponde a ningún requisito funcional ni caso de uso documentado; su inclusión se sustenta únicamente en la restricción de diseño RD-02 (acceso basado en roles).
2. La clase EvaluacionVeterinaria, con los atributos aptitudCruza y aptitudAdopcion, introduce un concepto de evaluación de aptitud no descrito explícitamente en ningún requisito funcional, solapándose parcialmente con RF-09. Se recomienda formular un requisito funcional adicional que sustente esta funcionalidad, o integrarla dentro de HistorialMedico para evitar sobre-especificación.

Verificaciones cruzadas entre perspectivas

1. **CU ↔ Clases UML:** el caso de uso CU-07 (Gestionar proceso de adopción) ejecuta directamente las operaciones aceptar() y rechazar() definidas en la clase Solicitud.
2. **Clases UML ↔ Procesos DFD:** la clase Solicitud es manipulada por el proceso P5 (Gestión de Solicitudes) del DFD Nivel 1, coincidiendo ambas representaciones en las mismas transformaciones (registrar, aceptar, rechazar).
3. **Estados UML ↔ CU:** los estados Pendiente, Aceptada y Rechazada del diagrama de máquina de estados de Solicitud corresponden exactamente a los pasos 3 a 6 del flujo normal de eventos de CU-07.
4. **Actividades UML ↔ CU:** el diagrama de actividades UML constituye la representación gráfica del flujo normal y del flujo alterno 4.1 de CU-01 (Registrar mascota).
5. **CU-03 ↔ Clases UML:** la relación de extensión («Extend») entre CU-03 (Cargar documento veterinario) y CU-02 (Registrar historial médico) corresponde a la asociación entre las clases HistorialMedico y DocumentoVeterinario en el diagrama de clases.

7.2. Tabla comparativa de los tres modelos

Conforme al Paso 5c de la guía, se presenta la comparación entre los tres tipos de modelos trabajados en las secciones 4, 5 y 6, evaluando la perspectiva que cubre cada uno, su notación, los requisitos que permite descubrir o validar, y sus limitaciones inherentes.

Criterio	Modelo de datos	Modelo funcional	Modelo de comportamiento
Perspectiva cubierta	Qué información gestiona el sistema: entidades, atributos y relaciones persistentes	Qué procesos y transformaciones realiza el sistema sobre los datos	Cómo cambian de estado las entidades y cómo interactúan los objetos a lo largo del tiempo
Notación utilizada	Diagrama Entidad-Relación (Crow's Foot) y Diagrama de Clases UML	DFD (Nivel 0, Nivel 1, Esencial) y Diagrama de Actividades UML	Máquina de Estados UML y Diagrama de Secuencia UML
Artefactos construidos	4.2 ER, 4.3 Clases UML	5.1-5.2 DFD, 5.3 Actividades	6.1 Estados, 6.2 Secuencia
Requisitos que ayuda a descubrir/validar	Complejidad de los atributos exigidos por cada RF (ej. RF-01: qué campos debe tener una mascota); consistencia de tipos de datos y restricciones (NOT NULL, UNIQUE); relaciones faltantes entre entidades mencionadas en distintos RF	Procesos faltantes o mal delimitados (ej. ausencia de un proceso de mensajería para RF-07, detectada en la matriz de trazabilidad); flujos de datos incompletos entre actores y almacenes; secuencia lógica de pasos de un CU	Estados o transiciones no contempladas en la redacción original del requisito (ej. qué ocurre si una solicitud es rechazada); condiciones de guarda faltantes en decisiones (alt); interacciones entre objetos no consideradas al redactar el RF en lenguaje natural
Limitaciones	No revela el comportamiento dinámico del sistema: una entidad bien modelada no garantiza que el proceso que la usa esté correctamente definido; tampoco expresa condiciones ni excepciones de negocio	No expresa el estado interno de las entidades a través del tiempo, solo el flujo de datos entre procesos; tampoco detalla la lógica de decisión interna de cada proceso (queda a nivel de caja negra)	No representa la estructura de almacenamiento de datos ni el flujo completo de información entre múltiples procesos del sistema; su alcance se limita a una entidad (estados) o a un escenario específico

			(secuencia), no a la totalidad del sistema
--	--	--	--

7.3. Conclusiones

1. Ninguna perspectiva, por sí sola, garantiza la completitud del SRS. El proceso de modelado evidenció que la perspectiva de datos puede definir correctamente un atributo (por ejemplo, estado en la clase Solicitud), pero solo la perspectiva de comportamiento revela los valores concretos que dicho atributo puede tomar y las condiciones que gatillan su cambio. Esta necesidad de triangulación confirma lo señalado por Pohl [2] respecto a que la validación de requisitos requiere verificar múltiples vistas del sistema de forma cruzada.

2. La perspectiva funcional actúa como mecanismo de detección de vacíos que las otras perspectivas no revelan. La construcción del DFD Nivel 1 permitió identificar que RF-07 (mensajería) carecía de un proceso dedicado, pese a que la clase `Mensaje` ya estaba correctamente definida en el modelo de datos. Esto demuestra que una entidad bien modelada no garantiza que el proceso que la utiliza haya sido correctamente delimitado, respaldando el principio de trazabilidad bidireccional exigido por IEEE/ISO/IEC 29148:2018 [1].

3. La perspectiva de comportamiento expone reglas de negocio implícitas que el lenguaje natural del requisito no siempre explicita. Al modelar la máquina de estados de Solicitud, fue necesario definir explícitamente qué ocurre cuando una solicitud es rechazada (retorno al estado Publicada), una regla que RF-05 sugiere, pero no detalla en su redacción original. Esto coincide con lo indicado por Harel [13], para quien el formalismo de statecharts permite hacer explícito el comportamiento reactivo de un sistema que de otra forma permanecería implícito en la especificación textual.

4. Las verificaciones cruzadas entre modelos son la principal herramienta para detectar tanto gaps como gold plating. El cruce entre casos de uso, clases UML y procesos DFD permitió identificar que la clase Administrador y la clase EvaluacionVeterinaria no cuentan con un requisito funcional que las sustente explícitamente (gold plating), mientras que RF-08, RF-12, RF-14, RF-15 y RF-17 carecen de un caso de uso formalizado (gap), pese a estar cubiertos en otras perspectivas. Sin la matriz de trazabilidad que integra las tres vistas, ambos tipos de inconsistencia habrían pasado inadvertidos.

5. La complementariedad entre datos, función y comportamiento responde a que cada perspectiva captura una dimensión distinta e irreductible del sistema. El modelo de datos responde a qué información existe; el modelo funcional, a cómo se transforma; y el modelo de comportamiento, a cuándo y bajo qué condiciones cambia. El SWEBOK v4.0 [7] enfatiza que esta triangulación de vistas es indispensable para lograr una especificación de requisitos completa, consistente y verificable, tal como lo exige el modelo de calidad de ISO/IEC 25010:2023 [6] aplicado a lo largo de todo este documento.

Anexo A – Tabla de atributos completa de todos los RF y RFN

Anexo B – Registro de defectos del SRS con correcciones aplicadas

Anexo C – Exportaciones originales de todos los diagramas (alta resolución)

Anexo D – Referencias IEEE y declaración de uso de IA

- [1] ISO/IEC/IEEE, “29148:2018 — Requirements engineering,” 2018, *IEEE, Piscataway, NJ, USA*. doi: 10.1109/IEEESTD.2018.8559686.
- [2] Klaus. Pohl, *Requirements engineering : fundamentals, principles, and techniques*. Springer, 2025.
- [3] M. A. Umar and K. Lano, “Advances in automated support for requirements engineering: a systematic literature review,” *Requir. Eng.*, vol. 29, no. 2, pp. 177–207, jun. 2024, doi: 10.1007/s00766-023-00411-0.
- [4] ISO/IEC/IEEE, “15288:2023 — System life cycle processes,” 2023.
- [5] ISO/IEC/IEEE, “12207:2017 — Systems and software engineering — Software life cycle processes,” 2017.
- [6] ISO/IEC, “25010:2023 Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model,” 2023.
- [7] H. Washizaki, *SWEBOK Guide v4.0*. IEEE Computer Society, 2024. [Online]. Available: <https://www.computer.org/education/bodies-of-knowledge/software-engineering>
- [8] P. M. Institute, *A Guide to the Project Management Body of Knowledge (PMBOK® Guide) – Seventh Edition and The Standard for Project Management*. Newtown Square, PA: Project Management Institute, 2021.
- [9] IREB, “CPRE Foundation Level Syllabus v3.1,” 2022. [Online]. Available: <https://www.ireb.org>
- [10] G. C. Oztekin and G. G. Menekse Dalveren, “Structured SRS for e-Government Services With Boilerplate Design and Interface,” *IEEE Access*, vol. 11, pp. 62906–62917, 2023, doi: 10.1109/ACCESS.2023.3287882.
- [11] E. Hull, K. Jackson, and J. Dick, *Requirements Engineering*. London: Springer London, 2011. doi: 10.1007/978-1-84996-405-0.
- [12] Leszek. Maciaszek, *Requirements analysis and system design*. Addison-Wesley, 2007.

- [13] D. Harel, “Statecharts: a visual formalism for complex systems,” *Sci. Comput. Program.*, vol. 8, no. 3, pp. 231–274, jun. 1987, doi: 10.1016/0167-6423(87)90035-9.